

Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №8

ШАБЛОНИ «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR»

Виконав
студент групи ІА–22:
Артамонов Дмитро Вікторович
Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Хід роботи 3
Короткі теоретичні відомості 3
Реалізація не менше 3-х класів відповідно до обраної теми. 7
Реалізація шаблону за обраною темою 8
Результат виконання патерну 11
Діаграма класів для паттерну 12
Висновок: 14

Тема: шаблони «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR».

Мета: ознайомитися з шаблонами проектування «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR» та набути практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи

..15 E-mail клієнт (singleton, builder, decorator, template method, interpreter, SOA)

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти

---

pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

Короткі теоретичні відомості

Шаблони роботи з БД при розробці корпоративних додатків: Існує набір сучасних шаблонів, які використовуються для розробників широкомасштабних корпоративних додатків - з кількістю користувачів понад 100 і великою кількістю взаємопов'язаних програм. Такі додатки вимагають окремих підходів до організації доступу до даних, оскільки кількість даних, що зберігаються, класів і клієнтів, що звертаються, зростає.

Шаблон «Active Record»: Поєднує дані й логіку в одному об'єкті, що представляє рядок БД. Широко використовується через простоту, але складність класу зростає зі збільшенням запитів, тому логіку часто виносять окремо.

Шаблон «Table Data Gateway»: Окремий клас для взаємодії з БД для кожного класу даних. Вся логіка запитів зосереджена в ньому, що підвищує гнучкість і тестованість. Часто абстрагується в базовий клас для зменшення дублювання. *Шаблон «Data Mapping»:* Перетворює об'єкти даних у формат, що приймається БД чи мережею. Маппер пов'язує властивості об'єкта з колонками таблиці, вирішуючи невідповідності типів.

Шаблон «Composite»

Призначення: Патерн "Composite" використовується для представлення і роботи з деревовидними структурами, де окремі об’єкти (листи) і групи об’єктів (вузли) обробляються однаково.

Проблема: Патерн "Composite" корисний для роботи з деревовидною структурою. Наприклад, продукти й коробки, які можуть містити інші коробки. Задача: обчислити загальну ціну замовлення, не знаючи наперед структури вкладеності.

Рішення: "Composite" уніфікує роботу з продуктами й коробками через спільний інтерфейс. Продукт повертає свою ціну, коробка обчислює суму вартості свого вмісту, незалежно від рівнів вкладеності.

Приклад з життя: Армія — структура у вигляді дерева: солдати, взводи, полки, армії. Накази передаються зверху вниз за ієрархією.

Переваги:

• Спрощує роботу зі складними структурами.

• Полегшує додавання нових компонентів.

• Створює загальнодизайнерське рішення.

Недоліки:

• Може бути складним для реалізації.

• Перебір вмісту може вимагати значних ресурсів.

Шаблон «Flyweight»

Призначення: Шаблон використовується для зменшення кількості об'єктів шляхом поділу загальних даних між екземплярами. Наприклад, у графічних системах або текстових редакторах.

Проблема: В іграх або додатках із великою кількістю однакових об'єктів (наприклад, кулі, частинки, текстові символи) обсяг оперативної пам’яті стає критичним. Кожен об'єкт містить багато даних, які можуть дублюватися.

Рішення: Розділяти стан об’єктів на:

- Внутрішній – загальні дані, які зберігаються в спільному об'єкті.

- Зовнішній – унікальні дані, пов’язані з конкретним контекстом

використання. Це дозволяє значно скоротити використання пам'яті.

Приклад з життя: У ветеринарній базі даних коти мають загальні (назва породи, текстура) і унікальні (ім’я, вік) дані. Загальні дані зберігаються окремо і використовуються спільно.

Переваги:

• Економія оперативної пам'яті.

• Уніфікація структури даних.

Недоліки:

• Зростає складність коду.

• Витрачається час на пошук спільних даних.

Шаблон «Intepreter»

Призначення: Шаблон використовується для подання граматики і створення інтерпретатора для мови (наприклад, скриптової), що дозволяє працювати з термінальними та нетермінальними символами у вигляді деревовидної структури.

Проблема: Необхідно автоматизувати розбір виразів або завдань, які часто змінюються, і реалізувати зрозумілу систему обчислення.

Рішення: Створюється абстрактне синтаксичне дерево, вузли якого представлені класами:

- Термінальний вираз – відповідає за базу рекурсії.

- Нетермінальний вираз – реалізує обробку підвиразів (рекурсивно). Кожен вузол виконує операцію розбору з урахуванням переданого контексту.

Приклад з життя: Припустимо потрібний автоматичний розбір поставленого завдання таким чином, що воно розбивається на прості та стає зрозумілим системі.

Переваги:

• Легка розширюваність граматики.

• Простота змін у способі обчислення виразів.

Недоліки:

• Ускладнення підтримки граматики з великою кількістю правил.

Шаблон «Visitor»

Призначення патерну: Патерн дозволяє вказувати операції над елементами без зміни їхньої структури. Групує однотипні операції, застосовувані до різних об'єктів.

Проблема: Уявіть програму з графами (вузли — міста, локації тощо). Завдання — реалізувати експорт графа в XML без змін у стабільному коді вузлів, але при цьому дозволити додавання нових операцій.

Рішення: Нову поведінку реалізують через окремий клас "Відвідувач". Об'єкти передають себе до методу відвідувача для виконання операцій. Це дозволяє додавати нові операції без змін у класах вузлів.

Приклад з життя: Компілятор, що генерує код для різних елементів синтаксису (виклики методів, умовні вирази). Кожен елемент передає себе відвідувачу, який виконує необхідні операції для генерації або перевірки коду.

Переваги:

• Додає нові операції без змін у коді об'єктів.

• Групує логіку, що застосовується до різних об'єктів.

Недоліки:

• Додає складність у реалізації.

• Необхідно підтримувати багато відвідувачів для різних операцій.

Реалізація не менше 3-х класів відповідно до обраної теми.

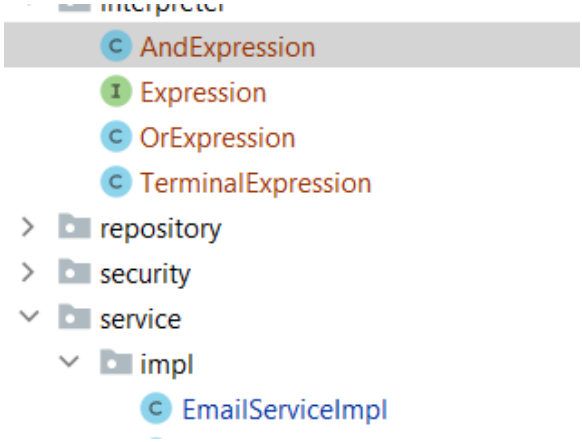


Рисунок 1 – структура класів

Expression:

Це базовий інтерфейс, який визначає метод interpret(String context). Він задає контракт для всіх виразів, які можуть бути інтерпретовані. У контексті шаблону "Інтерпретатор" цей інтерфейс є ядром, що дозволяє реалізувати різні типи логічних виразів.

TerminalExpression:

Цей клас представляє конкретний термінальний вираз, який виконує перевірку: чи містить переданий текст заданий ключовий термін.

OrExpression:

Цей клас відповідає за комбінування двох виразів за логічною операцією "АБО". Він об'єднує два підвирази, перевіряючи, чи виконується хоча б один із них. У цьому випадку він служить для створення складних виразів шляхом рекурсивного поєднання.

AndExpression:

Цей клас представляє логічну операцію "І" між двома виразами. Він інтерпретує результат лише тоді, коли обидва підвирази повертають true.

Реалізація шаблону за обраною темою

Кроки реалізації патерну:

```
public interface Expression {

    boolean interpret(String context);
}
```

Рисунок 2 – Expression

Клас Expression є інтерфейсом. Він визначає контракт для всіх типів виразів, які будуть використовуватися в системі. У цьому інтерфейсі є лише один метод: boolean interpret(String context);

- Метод interpret приймає на вхід рядок (context) і повертає логічне значення (true або false) залежно від реалізації конкретного виразу.

```
2 usages
public class OrExpression implements Expression {
    2 usages
    private Expression expr1;
    2 usages
    private Expression expr2;

    1 usage
    public OrExpression(Expression expr1, Expression expr2) {
        this.expr1 = expr1;
        this.expr2 = expr2;
    }

    5 usages
    @Override
    public boolean interpret(String context) {
        return expr1.interpret(context) || expr2.interpret(context);
    }
}
```

Рисунок 3 – OrExpression

Клас OrExpression реалізує інтерфейс Expression і представляє логічну операцію "АБО" (OR) у системі. Він дозволяє об'єднувати два вирази (expr1 і expr2) та перевіряти, чи виконується хоча б один із них.

Основні складові:

1. **Поля:**

   expr1 і expr2 — це два об'єкти типу Expression. Вони представляють підвирази, які потрібно об'єднати через операцію OR.

2. **Метод interpret(String context):**

   Виконує перевірку обох виразів.

   Повертає true, якщо хоча б один із виразів (expr1 або expr2) повертає true для переданого context.

```
public class AndExpression implements Expression {
    2 usages
    private Expression expr1;
    2 usages
    private Expression expr2;

    no usages
    public AndExpression(Expression expr1, Expression expr2) {
        this.expr1 = expr1;
        this.expr2 = expr2;
    }

    5 usages
    @Override
    public boolean interpret(String context) {
        return expr1.interpret(context) && expr2.interpret(context);
    }
}
```

Рисунок 4 – AndExpression

Клас AndExpression реалізує інтерфейс Expression і представляє логічну операцію "І" (AND) у системі. Він дозволяє об'єднати два вирази (expr1 і expr2) та перевірити, чи обидва вони виконуються для заданого контексту.

Опис класу AndExpression

Клас AndExpression реалізує інтерфейс Expression і представляє логічну операцію "І" (AND) у системі. Він дозволяє об'єднати два вирази (expr1 і expr2) та перевірити, чи обидва вони виконуються для заданого контексту.

Основні складові:

1. **Поля:**

   expr1 і expr2 — це два об'єкти типу Expression. Вони представляють підвирази, які мають виконатися одночасно.

2. **Метод interpret(String context):**

   Виконує перевірку обох виразів.

   Повертає true, якщо обидва вирази (expr1 і expr2) повертають true для переданого context.

   Повертає false, якщо хоча б один із виразів повертає false.

```
public class TerminalExpression implements Expression {
    2 usages
    private String data;

    2 usages
    public TerminalExpression(String data) {
        this.data = data;
    }

    5 usages
    @Override
    public boolean interpret(String context) {
        return context.contains(data);
    }
}
```

Рисунок 5 – TerminalExpression

Клас TerminalExpression реалізує інтерфейс Expression і представляє термінальний вираз у шаблоні **Interpreter**. Це найпростіший вид виразів, який

виконує конкретну перевірку для заданого контексту. У цьому випадку він перевіряє, чи містить контекст задане слово або фрагмент тексту.

Основні складові:

1. **Поле:**

   data — це рядок, який представляє ключове слово або фрагмент тексту, що буде використовуватися для перевірки в контексті.

2. **Метод interpret(String context):**

   Приймає рядок context, у якому виконується пошук.

   Повертає true, якщо контекст містить ключове слово data.

   Повертає false, якщо ключове слово відсутнє.

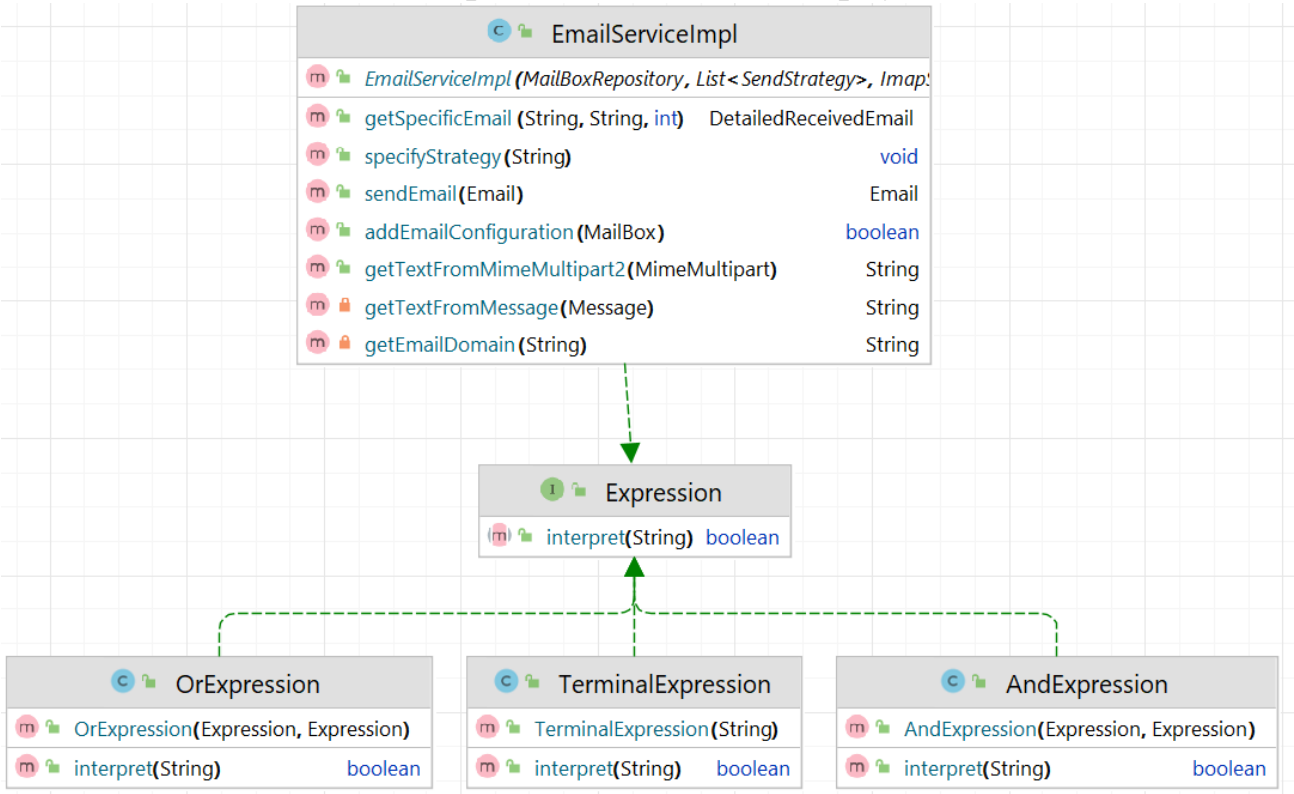
Результат виконання патерну


```
{
    "timeStamp": "2024-12-21T02:04:05.628892200",
    "code": 200,
    "httpStatus": "OK",
    "path": "/api/v1/read/qjsoq@ukr.net/Sent/3",
    "data": {
        "email": {
            "body": "\ntest email\r\n\r\n",
            "receiverEmail": "artamonovdima355@gmail.com",
            "senderEmail": "qjsoq@ukr.net",
            "subject": "Newsletter",
            "receivedDate": "2024-12-21T00:02:56.000+00:00",
            "important": true
        }
    }
}

{
    "timeStamp": "2024-12-21T02:05:18.901089900",
    "code": 200,
    "httpStatus": "OK",
    "path": "/api/v1/read/qjsoq@ukr.net/Sent/1",
    "data": {
        "email": {
            "body": "\nhello vol2\r\n\r\n",
            "receiverEmail": "kevinkenobi270@gmail.com",
            "senderEmail": "qjsoq@ukr.net",
            "subject": "hello vol1",
            "receivedDate": "2024-12-03T14:45:23.000+00:00",
            "important": false
        }
    }
}
```


Рисунок 6 – результат роботи

Діаграма класів для паттерну



1. Клас EmailServiceImpl

Призначення: EmailServiceImpl є сервісом, який забезпечує функціонал для роботи з електронними листами. Він взаємодіє з репозиторієм електронної пошти (MailBoxRepository), підтримує стратегії надсилання (SendStrategy) і реалізує різноманітні функції роботи з поштою.

2. Інтерфейс Expression

Призначення: Це базовий інтерфейс для всіх виразів, які будуть використовуватися для інтерпретації тексту.

Метод:

interpret(String context): Повертає boolean, що вказує, чи задовольняє переданий контекст умову виразу.

3. Клас OrExpression

Призначення: Використовується для об'єднання двох виразів за допомогою логічної операції "АБО" (OR).

Методи:

interpret(String context): Повертає true, якщо хоча б один із виразів (expr1 або expr2) повертає true для заданого контексту.

4. Клас TerminalExpression

Призначення: Реалізує термінальний вираз, який перевіряє, чи містить заданий контекст певне ключове слово.

Методи:

interpret(String context): Повертає true, якщо контекст містить задане ключове слово.

5. Клас AndExpression

Призначення: Використовується для об'єднання двох виразів за допомогою логічної операції "І" (AND).

Методи:

interpret(String context): Повертає true, якщо обидва вирази (expr1 і expr2) повертають true для заданого контексту.

Проблема, яку допоміг вирішити шаблон Interpreter:

Шаблон Interpreter допоміг вирішити проблему обробки і виконання виразів. Він дозволяє створити систему, яка розбирає вхідні вирази за допомогою чітко визначеної граматики та виконує відповідні дії. Цей шаблон забезпечує гнучкість у додаванні нових елементів і розширенні можливостей без значних змін у основній структурі коду. Він також дозволяє створювати ієрархічні структури для представлення складних виразів, де кожен компонент обробляється окремо.

Переваги застосування шаблону Interpreter:

По-перше, Interpreter забезпечує гнучкість у розширенні та модифікації системи. Оскільки кожен тип виразу чи операції реалізується окремим класом, додавання нових типів виразів або команд не вимагає змін в основному коді, що робить систему легше підтримуваною та розширюваною.

По-друге, шаблон дозволяє створювати ієрархічні структури, де кожен компонент може бути представлений як окремий об'єкт. Це дозволяє організувати складні вирази або правила в зручному для розуміння вигляді, де кожен елемент обробляється незалежно. Це спрощує розбір та виконання виразів, знижуючи складність коду.

Також, застосування шаблону допомагає в реалізації чіткої граматики для обробки виразів. Це корисно, коли потрібно точно визначити, як будуть трактуватися вхідні дані, знижуючи ймовірність помилок у виконанні.

Посилання на репозиторій проекту: https://github.com/qjsoq/Email-App

Висновок:
У цій лабораторній роботі я реалізував патерн Interpreter для Email-клієнта, що дозволило створити систему для визначення важливості листів. Кожна операція була представлена окремим класом, що полегшило додавання нових функцій без змін в основному коді. Це зробило систему більш гнучкою, легко масштабованою і зручною для подальшого розвитку.